МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил:
М.А. Романюк
студент группы ПО-13
Проверил:
А.А. Крощенко
доц. кафедры ИИТ,
05.03.2026

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Задание 1:

Вариант 9. Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

Выполнение:

Код программы

```
"""Модуль для системы заказов бургер-закусочной с использованием паттерна Builder."""

from abc import ABC, abstractmethod


class Order:
    """Готовый заказ."""

    def __init__(self):
        self.burger = None
        self.drink = None
        self.packaging = None
        self.total_price = 0

    def set_burger(self, burger):
        self.burger = burger

    def set_drink(self, drink):
        self.drink = drink

    def set_packaging(self, packaging):
        self.packaging = packaging

    def calculate_total(self):
        self.total_price = (
            (self.burger.price if self.burger else 0)
            + (self.drink.price if self.drink else 0)
            + (self.packaging.price if self.packaging else 0)
        )
        return self.total_price

    def show(self):
        print("\n" + "=" * 40)
        print("         ВАШ ЗАКАЗ")
        print("=" * 40)
        if self.burger:
            print(f"Бургер:   {self.burger.name} ({self.burger.category}) - {self.burger.price} BYN")
        if self.drink:
            print(f"Напиток:  {self.drink.name} ({self.drink.category}) - {self.drink.price} BYN")
        if self.packaging:
            print(f"Упаковка: {self.packaging.name} - {self.packaging.price} BYN")
        print("-" * 40)
```

```
        print(f"ИТОГОВАЯ СТОИМОСТЬ: {self.calculate_total()} BYN")
        print("=" * 40)


# компоненты заказа
class Burger:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category


class Drink:
    def __init__(self, name, price, category):
        self.name = name
        self.price = price
        self.category = category


class Packaging:
    def __init__(self, name, price):
        self.name = name
        self.price = price


# интерфейс строителя
class OrderBuilder(ABC):
    @abstractmethod
    def reset(self):
        pass

    @abstractmethod
    def add_burger(self, choice):
        pass

    @abstractmethod
    def add_drink(self, choice):
        pass

    @abstractmethod
    def add_packaging(self, choice):
        pass

    @abstractmethod
    def get_order(self):
        pass


# конкретный строитель
class FastFoodOrderBuilder(OrderBuilder):
    def __init__(self):
        self.reset()

    def reset(self):
        self.order = Order()
        return self

    def add_burger(self, choice):
        burgers = {
            "1": Burger("Classic (beef)", 8.50, "мясной"),
            "2": Burger("Chicken", 7.50, "куриный"),
            "3": Burger("Vegan (with veggies)", 6.80, "веганский"),
            "4": Burger("Double cheese", 11.00, "мясной"),
        }
```

```
        if choice in burgers:
            self.order.set_burger(burgers[choice])
        return self

    def add_drink(self, choice):
        drinks = {
            "1": Drink("Pepsi", 2.80, "холодный"),
            "2": Drink("Coca-Cola", 2.80, "холодный"),
            "3": Drink("Americano coffee", 3.50, "горячий"),
            "4": Drink("Black tea", 2.50, "горячий"),
            "5": Drink("Sprite", 2.80, "холодный"),
        }
        if choice in drinks:
            self.order.set_drink(drinks[choice])
        return self

    def add_packaging(self, choice):
        packagings = {"1": Packaging("С собой", 0.70), "2": Packaging("На месте", 0.00)}
        if choice in packagings:
            self.order.set_packaging(packagings[choice])
        return self

    def get_order(self):
        return self.order


# директор - делает готовые шаблоны заказов
class OrderDirector:
    def __init__(self, builder_class):
        # храним класс, а не объект, чтобы каждый раз создавать новый строитель
        self.builder_class = builder_class

    def make_vegan_takeaway_order(self):
        builder = self.builder_class()
        return builder.reset().add_burger("3").add_drink("4").add_packaging("1").get_order()

    def make_classic_dinein_order(self):
        builder = self.builder_class()
        return builder.reset().add_burger("1").add_drink("1").add_packaging("2").get_order()


def main():
    print("\n" + "=" * 40)
    print("    ДОБРО ПОЖАЛОВАТЬ В БУРГЕР-ЗАКУСОЧНУЮ!")
    print("=" * 40)

    builder = FastFoodOrderBuilder()

    print("\n--- БУРГЕРЫ ---")
    print("1. Classic (beef) - 8.50 BYN (мясной)")
    print("2. Chicken - 7.50 BYN (куриный)")
    print("3. Vegan (with veggies) - 6.80 BYN (веганский)")
    print("4. Double cheese - 11.00 BYN (мясной)")
    burger_choice = input("\nВыберите бургер (1-4): ")

    print("\n--- НАПИТКИ ---")
    print("1. Pepsi - 2.80 BYN (холодный)")
    print("2. Coca-Cola - 2.80 BYN (холодный)")
    print("3. Americano coffee - 3.50 BYN (горячий)")
    print("4. Black tea - 2.50 BYN (горячий)")
    print("5. Sprite - 2.80 BYN (холодный)")
    drink_choice = input("\nВыберите напиток (1-5): ")

    print("\n--- ТИП УПАКОВКИ ---")
    print("1. С собой - 0.70 BYN")
```

```
    print("2. На месте - 0.00 BYN")
    packaging_choice = input("\nВыберите тип упаковки (1-2): ")

    # собираем заказ
    order = (
        builder.reset().add_burger(burger_choice).add_drink(drink_choice).add_packaging(packaging_choice).g
et_order()
    )

    order.show()

    # пример готовых заказов через директора
    print("\n" + "=" * 40)
    print("    ГОТОВЫЕ ЗАКАЗЫ (ПРИМЕР РАБОТЫ ДИРЕКТОРА)")
    print("=" * 40)

    # передаём в директор класс, а не объект
    director = OrderDirector(FastFoodOrderBuilder)
    vegan_order = director.make_vegan_takeaway_order()
    vegan_order.show()

    classic_order = director.make_classic_dinein_order()
    classic_order.show()


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
========================================
         ВАШ ЗАКАЗ
========================================
Бургер:   Classic (beef) (мясной) - 8.5 BYN
Напиток:  Pepsi (холодный) - 2.8 BYN
Упаковка: С собой - 0.7 BYN
----------------------------------------
ИТОГОВАЯ СТОИМОСТЬ: 12.0 BYN
========================================


========================================
    ГОТОВЫЕ ЗАКАЗЫ (ПРИМЕР РАБОТЫ ДИРЕКТОРА)
========================================


========================================
         ВАШ ЗАКАЗ
========================================
Бургер:   Vegan (with veggies) (веганский) - 6.8 BYN
Напиток:  Black tea (горячий) - 2.5 BYN
Упаковка: С собой - 0.7 BYN
----------------------------------------
ИТОГОВАЯ СТОИМОСТЬ: 10.0 BYN
========================================
```

Задание 2:

Вариант 9. Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

Выполнение:

Код программы

```
class DigitalClock:
```

```
    def get_hours(self):
        pass

    def get_minutes(self):
        pass

    def get_time(self):
        pass


class AnalogClock:

    def __init__(self, hour_angle, minute_angle):
        # угол часовой стрелки от 12 часов (0-360)
        self.hour_angle = hour_angle % 360
        # угол минутной стрелки от 12 часов (0-360)
        self.minute_angle = minute_angle % 360

    def set_by_time(self, hours, minutes):
        # минутная: 360 гр / 60 = 6 гр на минуту
        self.minute_angle = (minutes * 6) % 360
        # часовая: 30 гр на час + 0.5 гр на минуту
        self.hour_angle = ((hours % 12) * 30 + minutes * 0.5) % 360

    def get_hour_angle(self):
        return self.hour_angle

    def get_minute_angle(self):
        return self.minute_angle


# адаптер
class ClockAdapter(DigitalClock):

    def __init__(self, analog_clock):
        self.analog_clock = analog_clock

    def get_hours(self):
        # 360 гр = 12 часов, значит 1 час = 30 гр
        hours = (self.analog_clock.get_hour_angle() / 30) % 12
        return int(hours)

    def get_minutes(self):
        # 360 гр = 60 минут, значит 1 минута = 6 гр
        minutes = (self.analog_clock.get_minute_angle() / 6) % 60
        return int(minutes)

    def get_time(self):
        hours = self.get_hours()
        minutes = self.get_minutes()
        return f"{hours:02d}:{minutes:02d}"


def main():
    print("=" * 50)
    print("ПРОЕКТ «ЧАСЫ» — ПАТТЕРН АДАПТЕР")
    print("=" * 50)
```

```
    # создаём аналоговые часы (хранят углы)
    analog = AnalogClock(90, 180)  # часовая на 90 гр (3ч) а минутная на 180 гр (30 мин)
    print(f"Аналоговые часы: hour_angle={analog.hour_angle}°,
minute_angle={analog.minute_angle}°")

    # оборачиваем в адаптер
    adapter = ClockAdapter(analog)

    print("\nЧерез адаптер:")
    print(f"  Часы:   {adapter.get_hours()} ч")
    print(f"  Минуты: {adapter.get_minutes()} мин")
    print(f"  Время:  {adapter.get_time()}")

    print("\n" + "-" * 50)
    print("Устанавливаем время 14:45 (2:45) через аналоговые часы:")
    analog.set_by_time(14, 45)
    print(f"  Углы: часовая={analog.hour_angle}°, минутная={analog.minute_angle}°")
    print(f"  Через адаптер: {adapter.get_time()}")

    print("\n" + "-" * 50)
    print("Демонстрация разных углов:")

    test_angles = [
        (0, 0),  # 12:00
        (90, 0),  # 3:00
        (180, 180),  # 6:30
        (270, 330),  # 9:55
    ]

    for hour_angle, minute_angle in test_angles:
        analog = AnalogClock(hour_angle, minute_angle)
        adapter = ClockAdapter(analog)
        print(f"  Углы ({hour_angle:3.0f}°, {minute_angle:3.0f}°) -> время
{adapter.get_time()}")


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
==================================================
ПРОЕКТ «ЧАСЫ» — ПАТТЕРН АДАПТЕР
==================================================
Аналоговые часы: hour_angle=90°, minute_angle=180°

Через адаптер:
  Часы:   3 ч
  Минуты: 30 мин
  Время:  03:30

--------------------------------------------------
Устанавливаем время 14:45 (2:45) через аналоговые часы:
  Углы: часовая=82.5°, минутная=270°
  Через адаптер: 02:45

--------------------------------------------------
Демонстрация разных углов:
  Углы (  0°,   0°) -> время 00:00
  Углы ( 90°,   0°) -> время 03:00
  Углы (180°, 180°) -> время 06:30
  Углы (270°, 330°) -> время 09:55
```

Задание 3:

Вариант 9. Шифрование текстового файла. Реализовать класс-шифровщик текстового файла с поддержкой различных алгоритмов шифрования. Возможные варианты шифрования: удаление всех гласных букв из текста, изменение букв текста на буквы, получаемые фиксированным сдвигом из алфавита (например, шифром буквы а будет являться буква д для сдвига 4 и т.д.), применение операции исключающее или с заданным ключом.

Выполнение:

Код программы

```
import os
from abc import ABC, abstractmethod


class EncryptionStrategy(ABC):
    @abstractmethod
    def encrypt(self, text: str) -> str:
        pass

    @abstractmethod
    def decrypt(self, text: str) -> str:
        pass


# стратегия 1: вырезаем гласные
class RemoveVowelsStrategy(EncryptionStrategy):
    def encrypt(self, text: str) -> str:
        vowels = "aeiouyaeёиоуыэюяAEIOUYAEЁИОУЫЭЮЯ"
        result = "".join(ch for ch in text if ch not in vowels)
        return result

    def decrypt(self, text: str) -> str:
        return text + " [расшифровка невозможна - гласные удалены безвозвратно]"


# стратегия 2: сдвиг букв (Цезарь)
class CaesarShiftStrategy(EncryptionStrategy):
    def __init__(self, shift: int = 4):
        self.shift = shift

    def encrypt(self, text: str) -> str:
        result = []
        for ch in text:
            if "а" <= ch <= "я":
                base = ord("а")
                new_pos = (ord(ch) - base + self.shift) % 32
                result.append(chr(base + new_pos))
            elif "А" <= ch <= "Я":
                base = ord("А")
                new_pos = (ord(ch) - base + self.shift) % 32
```

```
                result.append(chr(base + new_pos))
            elif "a" <= ch <= "z":
                base = ord("a")
                new_pos = (ord(ch) - base + self.shift) % 26
                result.append(chr(base + new_pos))
            elif "A" <= ch <= "Z":
                base = ord("A")
                new_pos = (ord(ch) - base + self.shift) % 26
                result.append(chr(base + new_pos))
            else:
                result.append(ch)
        return "".join(result)

    def decrypt(self, text: str) -> str:
        reverse_strategy = CaesarShiftStrategy(-self.shift)
        return reverse_strategy.encrypt(text)


# стратегия 3: XOR с ключом
class XorStrategy(EncryptionStrategy):
    def __init__(self, key: str = "secret"):
        self.key = key

    def encrypt(self, text: str) -> str:
        result = []
        key_len = len(self.key)
        for i, ch in enumerate(text):
            key_char = self.key[i % key_len]
            xor_result = ord(ch) ^ ord(key_char)
            result.append(chr(xor_result))
        return "".join(result)

    def decrypt(self, text: str) -> str:
        return self.encrypt(text)


class FileEncryptor:
    def __init__(self, strategy: EncryptionStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: EncryptionStrategy):
        self.strategy = strategy

    def encrypt_file(self, input_path: str, output_path: str = None):
        if output_path is None:
            name, ext = os.path.splitext(input_path)
            output_path = f"{name}_encrypted{ext}"

        with open(input_path, "r", encoding="utf-8") as f:
            text = f.read()

        encrypted = self.strategy.encrypt(text)

        with open(output_path, "w", encoding="utf-8") as f:
            f.write(encrypted)

        print(f"Файл зашифрован: {output_path}")
        return output_path
```

```python
    def decrypt_file(self, input_path: str, output_path: str = None):
        if output_path is None:
            name, ext = os.path.splitext(input_path)
            output_path = f"{name}_decrypted{ext}"

        with open(input_path, "r", encoding="utf-8") as f:
            text = f.read()

        decrypted = self.strategy.decrypt(text)

        with open(output_path, "w", encoding="utf-8") as f:
            f.write(decrypted)

        print(f"Файл расшифрован: {output_path}")
        return output_path


def interactive_mode():
    """Интерактивный режим - пользователь выбирает алгоритм и файл."""
    print("=" * 60)
    print("ПРОЕКТ «ШИФРОВАНИЕ» — ПАТТЕРН СТРАТЕГИЯ")
    print("=" * 60)

    print("\nВыберите алгоритм шифрования:")
    print("1. Удаление гласных")
    print("2. Сдвиг Цезаря")
    print("3. XOR")

    choice = input("\nВаш выбор (1-3): ")

    if choice == "1":
        strategy = RemoveVowelsStrategy()
        print("Выбрано: удаление гласных")
    elif choice == "2":
        shift = int(input("Введите сдвиг (например, 4): "))
        strategy = CaesarShiftStrategy(shift)
        print(f"Выбрано: сдвиг Цезаря на {shift}")
    elif choice == "3":
        key = input("Введите ключ (например, secret): ")
        strategy = XorStrategy(key)
        print(f"Выбрано: XOR с ключом '{key}'")
    else:
        print("Неверный выбор!")
        return

    encryptor = FileEncryptor(strategy)

    file_path = input("\nВведите путь к файлу для шифрования: ")
    if os.path.exists(file_path):
        encryptor.encrypt_file(file_path)
    else:
        print(f"Файл {file_path} не найден!")


def main():
    interactive_mode()
```

```
if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
============================================================
ПРОЕКТ «ШИФРОВАНИЕ» — ПАТТЕРН СТРАТЕГИЯ
============================================================

Выберите алгоритм шифрования:
1. Удаление гласных
2. Сдвиг Цезаря
3. XOR

Ваш выбор (1-3): 1
Выбрано: удаление гласных

Введите путь к файлу для шифрования: c:\Users\Maks\Desktop\1.txt
Файл зашифрован: c:\Users\Maks\Desktop\1_encrypted.txt
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python